

PCAM aplicado con OpenMP

Simulación gravitacional N-Body en 2D · Computación Paralela y Distribuida · UVG

Integrantes	Nicolas Concuá (23197) · Esteban Cárcamo (23016) · Diego López (23747)
Modalidad	Grupal (3 integrantes)
Lenguaje	C11 + OpenMP (gcc -O2 -std=c11 -fopenmp)
Fecha	4 de octubre de 2026

1. Problema y configuración experimental

Cada cuerpo tiene posición, velocidad y masa. En cada paso se acumula la aceleración que producen los demás cuerpos (gravedad con *softening* $\text{EPS2} = 1e-4$) y luego se integra con Euler semiimplícito ($\text{DT} = 1e-3$). Las tres versiones parten de las mismas condiciones iniciales: semilla 42 y un generador LCG propio, idéntico en macOS y Linux.

Tamaño elegido: $N = 20\,000$ cuerpos y 10 pasos, iguales en todas las comparaciones. Con el N sugerido (5 000) la secuencial tarda 0,6 s en este equipo, muy poco para medir speedup sin ruido; con 20 000 tarda 8,38 s. Cada configuración se ejecutó 3 veces y se reporta el promedio (`results.csv`), con el portátil conectado y en perfil de energía *performance*. El tiempo se mide con `omp_get_wtime()` y solo cubre la simulación, no la inicialización.

Hardware: Intel Core i9-13980HX con 24 núcleos (8 de rendimiento con *hyper-threading* y 16 de eficiencia), es decir, 32 hilos lógicos (`omp_get_num_procs() = 32`). Esta arquitectura híbrida pesa en el análisis de escalabilidad.

2. Diseño PCAM

P · Partición

Se descompone el dominio por cuerpos. La unidad mínima es **una interacción (i, j)**: la contribución de j a la aceleración de i . Hay $N(N-1) \approx 4 \cdot 10^8$ interacciones por paso. En la versión B la tarea es la fila i completa: lee todas las posiciones y solo escribe `ax[i]`, `ay[i]`. En la C la tarea es el par (i, j) con $i < j$, que por la tercera ley de Newton suma $+F$ sobre i y $-F$ sobre j ; hay la mitad de interacciones, pero cada una escribe en dos acumuladores. La integración es una tarea por cuerpo, totalmente independiente.

Dependencias. El paso $t+1$ necesita todas las posiciones del paso t , así que el ciclo de pasos es secuencial. Dentro de un paso, la fase de fuerzas debe terminar completa antes de integrar; si no, se mezclarían posiciones de dos instantes. Por eso hay dos ciclos y una barrera entre ellos.

C · Comunicación

En memoria compartida, comunicar es leer y escribir datos compartidos:

Dato	Fase de fuerzas	Integración	Riesgo
<code>b[] .x, y, mass</code>	Solo lectura, todos los hilos	Escribe solo el dueño de <i>i</i>	Ninguno: hay una barrera entre las dos fases
<code>ax[i], ay[i]</code> (B)	Escribe solo el dueño de la fila	Lee el dueño	Ninguno
<code>ax[j], ay[j]</code> (C)	Varios hilos escriben el mismo <i>j</i>	Lectura	Race condition
<code>dx, dy, r2,</code> <code>axi...</code>	Privadas (se declaran dentro del ciclo)	—	Ninguno

Hay dos puntos de sincronización por paso, que son las barreras implícitas de cada `omp for`. En la C se añade la combinación (reducción) de los acumuladores de cada hilo.

A · Aglomeración

Una interacción cuesta unas 20 operaciones de punto flotante: tratarla como tarea haría que el reparto dominara. Se aglomera en dos niveles: (1) todas las interacciones de un cuerpo forman una fila que se recorre secuencialmente y acumula en registros (`axi, ayi`); (2) varias filas forman un *chunk* de OpenMP. Se compararon tres tamaños de chunk (*default*, 8 y 64) con cada schedule (sección 4).

M · Asignación (mapping)

Las filas se reparten con `#pragma omp for schedule(runtime);` el schedule y el chunk llegan por línea de comandos y se aplican con `omp_set_schedule()`, así que se mide sin recompilar. En B cada fila cuesta lo mismo (N-1 interacciones). En C la fila *i* cuesta N-1-*i*: la carga es **triangular**. La integración tiene un costo uniforme y usa `schedule(static)`.

3. Estrategia OpenMP y race conditions

- **Una sola región paralela** para toda la simulación: los hilos se crean una vez. Cada hilo recorre el ciclo de pasos completo y se coordinan con las barreras implícitas de los dos `omp for` de cada paso.
- `default(none)` obliga a declarar el ámbito: los cuerpos y los acumuladores son `shared`, y los temporales se declaran dentro del ciclo, así que son privados.
- `omp single nowait` registra `omp_get_num_threads()`: el programa reporta los hilos que se usaron realmente, no solo los pedidos.

Race condition de la versión C. Si el hilo A procesa el par (3, 7) y el hilo B el (5, 7) a la vez, los dos ejecutan `ax[7] -= ...`. Es una secuencia leer-sumar-escribir: si se intercalan, se pierde una contribución. Con N = 20 000 son unas $4 \cdot 10^8$ actualizaciones por paso. Se evaluaron tres estrategias:

Estrategia	Resultado
<code>critical</code>	Serializaría las $4 \cdot 10^8$ actualizaciones de cada paso: quedaría por debajo de la secuencial. Se descartó en el diseño y no se implementó.
<code>atomic (--mode atomic)</code>	Correcta, pero son $4 \cdot 10^9$ operaciones atómicas en total, con contención por líneas de caché: 8,39 s (static) y 15,45 s (dynamic 64) con 8 hilos, igual o peor que la secuencial.
Acumuladores privados por hilo (elegida)	Cada hilo escribe en su propia fila de <code>acc_x/acc_y</code> sin sincronizar. Después de la barrera, el dueño de cada cuerpo suma las T copias, las pone a cero e integra en el mismo ciclo. Cuesta $O(T \cdot N)$ frente a $O(N^2)$: 0,862 s con 8 hilos.

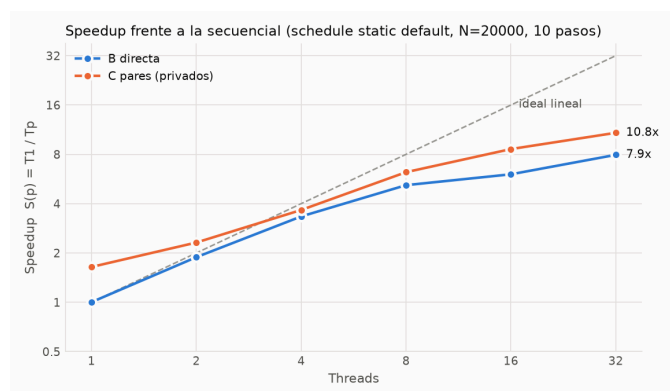
4. Resultados

$T_1 = 8,379$ s (versión A secuencial). $S(p) = T_1 / T_p$ y $E(p) = S(p) / p$, ambos contra la secuencial. Tabla obligatoria con B y C:

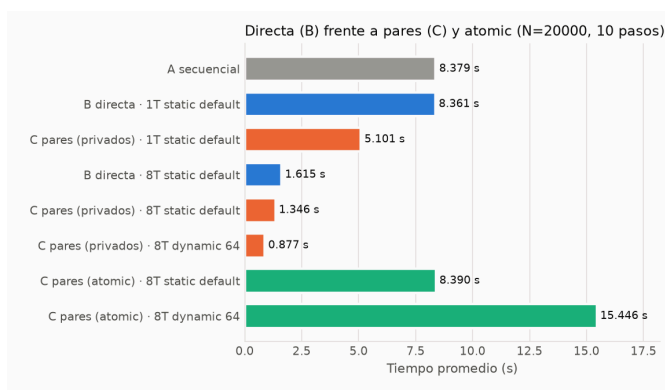
Threads	Schedule	Chunk	$\bar{T}_B$ (s)	$\bar{T}_C$ (s)	S / E B	S / E C
1	sequential	-	8.379	—	1.00 / 1.00	—
2	static	default	4.446	3.617	1.89 / 0.94	2.32 / 1.16
4	static	default	2.507	2.290	3.34 / 0.84	3.66 / 0.92
8	static	default	1.615	1.346	5.19 / 0.65	6.22 / 0.78
8	static	8	1.661	1.024	5.04 / 0.63	8.18 / 1.02
8	static	64	1.657	1.012	5.06 / 0.63	8.28 / 1.03
8	dynamic	8	1.533	0.873	5.47 / 0.68	9.60 / 1.20
8	dynamic	64	1.564	0.877	5.36 / 0.67	9.55 / 1.19
8	guided	default	1.598	1.450	5.24 / 0.66	5.78 / 0.72

Las tres configuraciones de chunk por schedule con 8 hilos (incluye *dynamic default*, *guided 8* y *guided 64*, que se añadieron para Aglomeración):

Schedule	Chunk	$\bar{T}_B$ (s)	S / E B	$\bar{T}_C$ (s)	S / E C
static	default	1.615	5.19 / 0.65	1.346	6.22 / 0.78
static	8	1.661	5.04 / 0.63	1.024	8.18 / 1.02
static	64	1.657	5.06 / 0.63	1.012	8.28 / 1.03
dynamic	default	1.539	5.45 / 0.68	0.862	9.72 / 1.22
dynamic	8	1.533	5.47 / 0.68	0.873	9.60 / 1.20
dynamic	64	1.564	5.36 / 0.67	0.877	9.55 / 1.19
guided	default	1.598	5.24 / 0.66	1.450	5.78 / 0.72
guided	8	1.598	5.24 / 0.66	1.434	5.84 / 0.73
guided	64	1.652	5.07 / 0.63	1.473	5.69 / 0.71



Speedup con static default; 16 y 32 hilos son puntos extra.



B frente a C (acumuladores privados y atomic).

Corrección numérica. Todas las corridas dan el checksum 21071.127819. Contra el estado final de la secuencial, la diferencia máxima de B es 0 (idéntica bit a bit, porque suma en el mismo orden) y la de C es $1,2 \cdot 10^{-16}$ (cuerpo 5057, vy). Esa diferencia sale de que en C cambia el orden de las sumas de punto flotante;

es redondeo y no una divergencia física. Las capturas de compilación, ejecución y hardware están en [docs/evidencia.pdf](#) y en [evidence/](#).

5. Preguntas de análisis

1. La unidad mínima es una interacción (i, j), de unas 20 operaciones. Como tarea individual habría $4 \cdot 10^8$ repartos por paso, y el costo de asignarlas superaría al cálculo. Aglomerarlas por fila deja tareas de 20 000 interacciones. Con eso, incluso *dynamic* con chunk 1 (el *default* de libgomp) solo hace $2 \cdot 10^5$ repartos en total: en C, *dynamic default* (0,862 s) y *dynamic 64* (0,877 s) quedan dentro del ruido.
2. Se comparten las posiciones y masas (solo lectura en la fase de fuerzas), el arreglo de cuerpos (escritura en la integración, pero cada cuerpo lo escribe solo su dueño y las fases están separadas por barreras) y los acumuladores. En B, `ax[i]` lo escribe un único hilo. En C, `ax[j]` recibe escrituras de muchos hilos y es lo único que necesita protección; se resolvió con copias privadas por hilo.
3. En B el chunk casi no influye (de 1,533 a 1,661 s, un 8 % entre el mejor y el peor): la carga es uniforme y las filas ya son tareas grandes. En C es decisivo con *static*: el bloque *default* le da al primer hilo las filas más caras (1,346 s), y con chunk 8 o 64 el reparto pasa a ser cíclico y equilibra el triángulo (1,024 y 1,012 s). Chunks pequeños dan más balanceo; ninguno de los probados añadió un overhead que se note, porque cada fila ya agrupa miles de interacciones.
4. **dynamic** fue el mejor en las dos versiones: 1,533 s en B (chunk 8) y 0,862 s en C (chunk *default*). En C, *static default* y *guided* quedan en el mismo rango (1,346 y 1,450 s), muy lejos de *dynamic*, porque el primer bloque de *guided* también es de unas N/T filas y contiene las más caras. La cota teórica del desbalance de *static* en C (el primer bloque concentra $1 - (1 - 1/p)^2$ del trabajo) da speedups máximos de 1,33, 2,29 y 4,27 para 2, 4 y 8 hilos, frente a 1,41, 2,23 y 3,79 medidos respecto a C con un hilo (con 2 hilos la supera un poco por la variación del turbo entre corridas). Que queden tan cerca confirma que el cuello de botella es el balanceo. En B, aunque la carga es uniforme, *dynamic* le gana a *static* por un 5 %: los núcleos son heterogéneos (P y E) y el reparto dinámico compensa a los hilos que caen en núcleos lentos.
5. Al procesar cada par una sola vez, dos hilos pueden actualizar a la vez `ax[j]` del mismo *j* (y `ax[i]`, que también reciben las filas anteriores). Se resolvió con acumuladores privados por hilo y una reducción posterior sin sincronización (sección 3).
6. No. C hace la mitad de interacciones, pero con un hilo es 1,64× más rápida, no 2×, porque cada par escribe en memoria (`my_ax[j]`) en lugar de acumular solo en registros. La variante atomic, con el mismo número de operaciones, fue la más lenta de todas (8,39–15,45 s) por el costo y la contención de las operaciones atómicas. Con *dynamic 64* fue peor que con *static*: los hilos procesan filas vecinas al mismo tiempo, recorren los mismos *j* y se disputan las mismas líneas de caché. Menos operaciones solo se traduce en menos tiempo si la sincronización y el desbalance que introduce la optimización no se comen la ganancia.
7. Con *static default*, B da S/E = 1,89/0,94, 3,34/0,84 y 5,19/0,65 para 2, 4 y 8 hilos, y C da 2,32/1,16, 3,66/0,91 y 6,23/0,78 (C supera E = 1 porque se compara contra una secuencial que hace el doble de trabajo; respecto a sí misma, con *dynamic* logra 5,92× con 8 hilos). B deja de escalar linealmente a partir de 4 hilos y la caída se acentúa en 8: el turbo baja cuando hay más núcleos activos, y las barreras y el reparto pesan más. Pasados 8 hilos se agotan los núcleos de rendimiento: con 16 hilos E cae a 0,38 y con 32, a 0,25.
8. No. Con la métrica de Karp-Flatt, el speedup de B con 8 hilos implica una fracción no paralelizable efectiva de $\approx 7,7$ %. Según Amdahl, con 64 núcleos el speedup sería $\approx 10,9$: 2,1 veces el de 8 hilos, no 8 veces. Las mediciones ya lo muestran: pasar de 8 a 32 hilos solo dio 1,53× en B y 1,74× en C. En una máquina de 64 núcleos homogéneos, además, cada hilo recibiría solo unas 312 filas con N = 20 000, y las

dos barreras por paso pesaría más. Para aprovecharlos habría que crecer el problema (Gustafson), no solo los núcleos.

6. Conclusiones

- La partición por filas es correcta y suficiente para B: sin sincronización explícita, el resultado es idéntico bit a bit a la secuencial y llega a 5,47× con 8 hilos (dynamic).
- Explotar la simetría reduce el trabajo a la mitad, pero introduce una race condition y una carga triangular. Los acumuladores privados resuelven la primera sin costo apreciable. El mapping (*dynamic* o *static* con chunk pequeño) resuelve la segunda: C pasa de 1,346 a 0,862 s, 1,78× más rápida que la mejor B.
- La sincronización fina sale cara: `atomic` hace que la versión con menos operaciones sea más lenta que la secuencial.
- La escalabilidad la limitan el desbalance, la fracción serial efectiva y el hardware híbrido. Con más núcleos no se espera un speedup proporcional con este N.